13 · Testing for ML Systems

> **Goal:** After this chapter you can (a) lay out a test pyramid for data, model, pipeline and serving with a time budget per layer, (b) write statistical, behavioural and slice-based tests that fail only when something real changed, and (c) test pipelines and serving for idempotency, load and degradation before users find the problem.

Table of Contents

- 13.1 The ML Test Pyramid and Test Strategy
- 13.2 Data Tests: Schema, Distribution, Leakage and Label Tests
- 13.3 Model Tests: Performance Floors, Calibration, Regression and Determinism
- 13.4 Behavioural Testing: Invariance, Directional and Metamorphic Tests
- 13.5 Slice-Based Testing, Error Analysis and Fairness-Aware Tests
- 13.6 Pipeline, Serving, Load and Production Testing

13.1 The ML Test Pyramid and Test Strategy

Conceptual Deep Dive

Testing an ML system differs from testing a service in four ways: behaviour is *learned* rather than written, oracles are *statistical* (there is no single correct output, only an acceptable distribution), failures are *silent* (the code runs and returns plausible numbers), and tests are *probabilistic* (a check can fail by chance). Module 05 section 2 (legacy) lists what to test; this chapter organises it into a **test pyramid** with a cost budget per layer.

The layers, from cheap and frequent to expensive and rare: **unit** tests of transforms and feature code (milliseconds); **data** tests of schema, distribution and leakage (13.2); **model** tests for floors, calibration and regression (13.3); **behavioural** and **slice** tests (13.4, 13.5); **pipeline/integration** tests on a tiny dataset (13.6); **serving/contract** tests; **system** tests with shadow traffic; and **production** checks, where monitors are continuous tests (see chapter 8). Give each layer a time budget: a PR gate in minutes (unit, data on a sample, smoke-train on a few thousand rows), a nightly run in tens of minutes (full model and slice suite), a release gate in hours (load, soak, replay).

For a rubric, use the *ML Test Score* paper (Breck et al., 2017): a checklist of tests and monitors grouped as data, model, infrastructure and monitoring, scored to place a team on a maturity ladder. It works as a self-assessment and a backlog generator; do not treat the score as a certification.

Flaky-test economics. A flaky test costs more than a missing one because the team learns to ignore red builds. Control seeds, use tolerance bands instead of exact equality, and write **statistical assertions** that fail only when a metric is below a floor *with confidence* (code below). A test at p=0.05 fails one run in 20 by design, so never gate on an uncorrected significance test across many checks. Quarantine flaky tests with an expiry date and an owner so quarantine does not become deletion.

Define "done" for a model change: unit and data tests green, model suite above floors against the golden set and the champion, slice floors held, behavioural tests unchanged, pipeline idempotency verified, a latency and size budget met, and the change linked to a registry record (see 4.15). The author owns the tests for their change; the platform owns the runners.

Edge cases: a test passes on CPU and fails on GPU due to numerics (use tolerances, pin the CI device); the CI sample is no longer representative after a data change (refresh it on a schedule, stratified); and a statistical test fails by design at its alpha level.

Cross-Connection Scenarios

- Data and schema validation in training pipelines is detailed in 13.2 and Module 02 section 1 (legacy).
- Monitors as production tests continue in chapter 8 and 8.17.
- Fairness checks are slice tests, see 13.5 and chapter 17.
- Promotion gates in the registry consume these results, see 4.15.

Production Anti-Patterns

- Only unit tests, no data or model tests: the pipeline is green while a unit-change in the data halves model quality.
- Test data copied from the training set: floors always pass and give false confidence.
- Loose thresholds that never fail: the suite becomes ceremony, and real regressions ship with a green check mark.

Minimal Working Code

A test-layer registry maps tests to layers and a runner fails the build when a layer exceeds its time budget; the clock is injected so the demo is deterministic.

```
import time
REGISTRY, T = [], [0.0]                    # T is a fake clock so the demo is deterministic

def layer(name):
    def deco(fn):
        REGISTRY.append((name, fn)); return fn
    return deco

@layer("pr")
def test_transform_unit():
    T[0] += 1.0; assert 2 * 3 == 6

@layer("pr")
def test_schema_sample():
    T[0] += 2.0; assert {"a", "b"} <= {"a", "b", "c"
}

def run_layer(name, budget_s, clock=lambda: T[0]):
    start, failures = clock(), []
    for lname, fn in REGISTRY:
        if lname == name:
            try:
                fn()
            except AssertionError as e:
                failures.append((fn.__name__, str(e)))
    elapsed = clock() - start
    if failures or elapsed > budget_s:
        raise RuntimeError(f"layer {name}: failures={failures} elapsed={elapsed:.1f}s budget={budget
_s}s")
    return elapsed

print("pr layer ok in", run_layer("pr", 5.0), "s")
@layer("pr")
def test_slow_new():
    T[0] += 4.0
try:
    run_layer("pr", 5.0)
```

```
except RuntimeError as e:
    print("blocked:", e)
```

A statistical assertion fails a test only when the metric is below a floor with confidence, using the lower bootstrap bound of AUC.

```
import numpy as np

def auc(s, y):
    r = np.empty(len(s)); r[np.argsort(s, kind="stable")] = np.arange(1, len(s) + 1)
    n1 = int(y.sum()); n0 = len(y) - n1
    return (r[y == 1].sum() - n1 * (n1 + 1) / 2) / (n1 * n0)

def assert_metric_at_least(scores, labels, floor, alpha=0.05, n_boot=300, seed=0):
    rng = np.random.default_rng(seed); n = len(labels); vals = []
    while len(vals) < n_boot:
        i = rng.integers(0, n, n)
        if 0 < labels[i].sum() < n:
            vals.append(auc(scores[i], labels[i]))
    lower = float(np.quantile(vals, alpha))                # one-sided lower bound
    assert lower >= floor, f"AUC lower bound {lower:.3f} < floor {floor} (point {auc(scores, l
abels):.3f})"

rng = np.random.default_rng(0)
y = rng.integers(0, 2, 800)
strong = y + rng.normal(0, 0.8, 800)
assert_metric_at_least(strong, y, floor=0.70)              # passes with confidence
weak = 0.25 * y + rng.normal(0, 0.8, 800)
try:
    assert_metric_at_least(weak, y, floor=0.65)
except AssertionError as e:
    print("fails correctly:", e)
```

Interview Angle

Q: How is testing an ML system different from testing a service? Give the layers and what each catches. **A:** Behaviour is learned, oracles are statistical and failures are silent. Unit tests catch transform bugs; data tests catch schema, unit and leakage problems; model tests catch floor, calibration and regression failures; behavioural and slice tests catch logic and cohort failures; pipeline tests catch idempotency and wiring bugs; serving tests catch contract and latency issues; production monitors catch drift. Trade-off: each layer costs time, so budget the PR gate in minutes. Measure: escaped defects per layer and gate duration.

Q: Your CI is flaky because of model tests. Fix it without deleting tests. **A:** Find the noise source first: seeds, data order, device numerics, small samples. Pin seeds and device, replace exact equality with tolerances, and replace point thresholds with bootstrap bounds so tests fail only with confidence. Quarantine remaining flaky tests with an owner and expiry. Trade-off: tolerance bands can hide small regressions, so keep a slower nightly test with tighter intervals. Measure: flake rate per test over 100 runs, targeted below 1%.

13.2 Data Tests: Schema, Distribution, Leakage and Label Tests

Conceptual Deep Dive

Data tests are the highest-yield layer because most production incidents start in data (see Module 11 (legacy)). **Schema and contract tests** check types, nullability, enums, ranges, uniqueness and referential integrity, and they run at three points: in the producer's CI (the contract owner), at ingestion (before bad data lands) and pre-train (before compute is spent). **Volume and freshness tests** catch half-loaded partitions, empty days and duplicates: compare row counts with a trailing baseline and assert the partition watermark.

Distribution tests compare current data with a reference: PSI (population stability index), the KS statistic for continuous features, and chi-square for categoricals. Pair each with an *effect-size* threshold, since with millions of rows a p-value flags trivial shifts. Common rules of thumb for PSI are below 0.1 stable and above 0.25 significant; treat them as starting points to calibrate on your own history. A special case is **train/serve parity**: recompute features offline for logged requests and compare with the values the service actually used.

Leakage tests flag features with timestamps after the prediction time, features derived from the label (suspiciously high single-feature AUC), and entity overlap between train and test. **Label tests** check class balance against the expected prior, label-delay maturity (no rows younger than the label window), annotator agreement where humans label (see 2.16), and the label-flip rate between snapshots.

Tool	Fits when
Great Expectations	declarative suites, data docs, many sources
Pandera	typed dataframe schemas inside Python code
dbt tests	the data lives in a warehouse and transforms are SQL
Deequ-style metrics on Spark	very large tables, metric history
Custom code	semantic checks no library expresses (units, joins, domain rules)

Custom code wins when the check is domain semantic or when the dependency cost outweighs the check. Decide which tests **block** (schema break, empty partition, leakage, label maturity) and which **warn** (mild drift): a blocking test must be rare and trustworthy, and a warn test needs an owner and review cadence.

Edge cases: validation passes on a sample but fails on the full partition (sampling skew); tests that hard-code today's distribution fail every holiday (use seasonal baselines); and a nullable column silently becomes all-null, which a type check does not see.

Cross-Connection Scenarios

- Contracts and Pydantic/Great Expectations basics are in Module 02 section 1 (legacy).
- Splits, leakage taxonomy and sampling are in 2.17.
- Drift monitors that reuse PSI and KS in production are in chapter 8.
- Label maturity and delays are in 2.16.

Production Anti-Patterns

- Log-and-continue validation: failures are recorded but training proceeds, and a corrupted model reaches the registry.
- Thresholds copied from a blog post: they fire constantly or never, and the team disables the suite.
- Testing only schema, not semantics: dollars become cents with the dtype unchanged, and every downstream metric shifts silently.

Minimal Working Code

A minimal expectation runner checks schema, null rate, range, uniqueness and allowed values, returning a structured report, plus an entity-overlap check between train and test.

```
def is_null(v):
    return v is None or (isinstance(v, float) and v != v)

def run_expectations(table, spec):
    fails = []
    for col, r in spec.items():
        if col not in table:
            fails.append((col, "missing_column", "")); continue
        vals = table[col]
        rate = sum(map(is_null, vals)) / len(vals)
        present = [v for v in vals if not is_null(v)]
        if rate > r.get("max_null", 0.0):
            fails.append((col, "null_rate", f"{rate:.3f}"))
        if "type" in r and not all(isinstance(v, r["type"]) for v in present):
            fails.append((col, "type", str(r["type"])))
        if "min" in r and any(v < r["min"] for v in present):
            fails.append((col, "min", str(min(present))))
        if "max" in r and any(v > r["max"] for v in present):
            fails.append((col, "max", str(max(present))))
        if r.get("unique") and len(set(present)) != len(present):
            fails.append((col, "unique", "duplicates"))
        if "allowed" in r and not set(present) <= r["allowed"]:
```

```
            fails.append((col, "allowed", str(set(present) - r["allowed"])))
    return fails

def entity_overlap(train_ids, test_ids):
    return sorted(set(train_ids) & set(test_ids))

spec = {"id": {"unique": True, "type": int}, "amount": {"typ
e": (int, float), "min": 0, "max": 10_000, "max_null": 0.01},
        "currency": {"allowed": {"USD", "EUR"}}}
table = {"id": [1, 2, 3, 3], "amount": [10.0, 25.5, 1_500_000.0, None], "currenc
y": ["USD", "EUR", "USD", "CAD"]}
for f in run_expectations(table, spec):
    print(f)
print("overlap:", entity_overlap([1, 2, 3], [3, 4]))
assert len(run_expectations(table, spec)) == 4 and entity_overlap([1, 2, 3], [3, 4]) == [3]
```

A parity test compares offline-recomputed features with logged serving features using PSI and the KS statistic with per-feature tolerances.

```
import numpy as np

def psi(ref, cur, bins=10, eps=1e-6):
    edges = np.unique(np.quantile(ref, np.linspace(0, 1, bins + 1)))
    edges[0], edges[-1] = -np.inf, np.inf
    r = np.clip(np.histogram(ref, edges)[0] / len(ref), eps, None)
    c = np.clip(np.histogram(cur, edges)[0] / len(cur), eps, None)
    return float(((c - r) * np.log(c / r)).sum())

def ks_stat(a, b):
    a, b = np.sort(a), np.sort(b)
    grid = np.concatenate([a, b])
    return float(np.abs(np.searchsorted(a, grid, side="right") / len(a)
                        - np.searchsorted(b, grid, side="right") / len(b)).max())

def parity_test(offline, serving, tol):
    report = {}
    for feat, t in tol.items():
        p, k = psi(offline[feat], serving[feat]), ks_stat(offline[feat], serving[feat])
        report[feat] = {"psi": round(p, 4), "ks": round(k, 4), "ok": p <
= t["psi"] and k <= t["ks"]}
    return report

rng = np.random.default_rng(0)
a = rng.normal(50, 10, 5000)
offline = {"age": a, "amount_usd": rng.lognormal(3, 1, 5000)}
serving = {"age": a + rng.normal(0, 0.01, 5000), "amount_usd": offline["amount_u
sd"] * 100}   # cents logged
rep = parity_test(offline, serving, {f: {"psi": 0.1, "ks": 0.05} for f in offline})
print(rep)
assert rep["age"]["ok"] and not rep["amount_usd"]["ok"]
```

Interview Angle

Q: Design the data tests for a fraud model's training pipeline and say which block training versus warn. **A:** Block: schema/type breaks, empty or partial partitions, duplicated keys, future timestamps, label-derived features, train/test entity overlap, and immature labels. Warn: moderate PSI/KS drift, class-prior movement within a band, mild null-rate increases. Run schema at ingestion and pre-train, leakage and overlap at split time. Trade-off: too many blockers stall retraining, so each blocker must have a precise, low false-positive rate. Measure: blocker false-positive rate and incidents caught per quarter.

Q: A unit change (dollars to cents) shipped upstream. Which test would have caught it and where should it run? **A:** A semantic range check (max amount) and a distribution test with an effect-size threshold, run in the producer's CI and at ingestion, plus a train/serve parity test downstream. Schema tests alone pass because the dtype is unchanged. Trade-off: range bounds need maintenance as the business grows. Measure: time-to-detect from deploy; target is the producer's CI, minutes instead of days.

13.3 Model Tests: Performance Floors, Calibration, Regression and Determinism

Conceptual Deep Dive

Model tests answer "is this artifact allowed to be promoted?" They run against a frozen **golden** evaluation set (see 2.17) built once, versioned, deduplicated and kept out of training. **Performance floors** are tested two ways: an absolute floor (the metric's confidence bound exceeds a business minimum) and a relative test against the champion with a margin. Use intervals, not point values (see 3.15), so the test fails on real regressions and not on bootstrap noise.

Calibration tests matter wherever probabilities feed thresholds, costs or risk limits. Compute the Brier score and the expected calibration error (ECE): bin predictions, compare mean predicted probability with observed frequency per bin, and weight by bin size. Test ECE against a budget, and test *threshold-specific* precision and recall because the deployed decision is a threshold, not an AUC.

Regression tests are a curated set of previously failed cases (from incidents, error analysis and support tickets) that must stay fixed; add to it every time production teaches something. **Robustness basics** check noise injection, missing-value injection (each feature set to missing, see the interview question below), out-of-range values and schema-valid but absurd inputs (age 400); the expectation is bounded, sensible output or a handled error, not a crash or NaN.

Determinism and serialization. Same seed and data should produce the same output within a stated tolerance (see 3.16). A **round-trip test** saves the model artifact, loads it, and asserts predictions equal the original within tolerance; it catches preprocessing that was not serialized with the model, a classic cause of skew. Test the serialized artifact the service will load, not the in-memory object. Pickle is acceptable only for trusted artifacts in a demo; prefer safer formats (ONNX, framework-native safe tensors, skops-style) for anything crossing a trust boundary (see chapter 10).

Budgets as tests. Model size, p95 inference latency on reference hardware and memory are assertions with thresholds from the serving SLO, so a heavier model fails in CI and not in production.

Edge cases: the model passes the aggregate floor while one cohort collapses (see 13.5); a round trip changes predictions because a scaler stayed in the notebook; and the golden set is contaminated by training data, which makes the floor meaningless (check ids and near-duplicates).

Cross-Connection Scenarios

- Promotion gates and model cards consume these results, see 4.15.
- Calibration after resampling ties to 2.17.
- Latency budgets reuse the serving SLOs from 6.17.
- Per-cohort floors are the subject of 13.5.

Production Anti-Patterns

- A single global AUC floor: a mobile or new-market cohort collapses while the average holds.
- No regression set: old bugs return because nothing remembers them.
- Testing only the in-memory model: the serialized artifact differs (missing preprocessing, version mismatch) and the failure appears in production.

Minimal Working Code

A reliability-bin calculator and ECE, with a test that fails when ECE exceeds its budget.

```
import numpy as np

def reliability(p, y, bins=10):
    idx = np.minimum((p * bins).astype(int), bins - 1)
    return [(b / bins, int((idx == b).sum()), float(p[idx == b].mean()), float(y[idx == b].mean()))
            for b in range(bins) if (idx == b).any()]

def ece(p, y, bins=10):
    return sum(cnt / len(p) * abs(conf - acc) for _, cnt, conf, acc in reliability(p, y, bins))
```

```
def assert_ece_below(p, y, budget):
    value = ece(p, y)
    assert value <= budget, f"ECE {value:.3f} > budget {budget}"

rng = np.random.default_rng(0)
p = rng.beta(2, 5, 20000)
y = (rng.random(20000) < p).astype(float)               # calibrated by construction
assert_ece_below(p, y, 0.02)
logit = np.log(p / (1 - p))
over = 1 / (1 + np.exp(-2.5 * logit))                   # overconfident version of the same ranking
try:
    assert_ece_below(over, y, 0.02)
except AssertionError as e:
    print("caught:", e)
```

A serialization round-trip test asserts predictions are equal within tolerance and enforces a latency budget.

```
import os, pickle, tempfile, time
import numpy as np

class Scorer:
    """Preprocessing lives inside the artifact, so the round trip preserves behaviour.&quo
t;"""
    def __init__(self, w, mean, std):
        self.w, self.mean, self.std = w, mean, std
    def predict(self, X):
        return 1 / (1 + np.exp(-(((X - self.mean) / self.std) @ self.w)))

rng = np.random.default_rng(0)
X = rng.normal(5, 2, size=(1000, 4))
model = Scorer(np.array([0.5, -0.2, 0.1, 0.3]), X.mean(0), X.std(0))

def test_round_trip_and_latency(model, X, atol=1e-12, budget_ms=50.0):
    with tempfile.TemporaryDirectory() as d:
        path = os.path.join(d, "model.pkl")
        with open(path, "wb") as f:
            pickle.dump(model, f)          # demo only: pickle runs code on load; use safer formats acros
s trust boundaries
        with open(path, "rb") as f:
            loaded = pickle.load(f)
    assert np.allclose(model.predict(X), loaded.predict(X), atol=atol), "round trip changed predicti
ons"
    times = []
    for _ in range(20):
        t = time.perf_counter(); loaded.predict(X); times.append((time.perf_counter() - t) * 1000)
    assert float(np.median(times)) <= budget_ms, f"median latency {np.median(times):.2f} ms over
budget"

test_round_trip_and_latency(model, X)
print("round trip and latency budget ok")
```

Interview Angle

Q: What goes in a model acceptance test suite before registry promotion? **A:** Absolute and champion-relative performance floors with intervals on a frozen golden set; slice floors; calibration (Brier/ECE) and threshold-specific precision/recall; a regression set of past failures; robustness with missing and out-of-range inputs; a serialization round-trip of the real artifact; and size, latency and memory budgets. Trade-off: a long suite delays promotion, so keep the heavy checks nightly. Measure: escaped regressions per quarter and suite runtime.

Q: How do you test that a model handles missing features sensibly? **A:** Inject missingness one feature at a time and in realistic joint patterns (an upstream outage drops a whole group), then assert outputs stay finite, within bounds, and degrade gracefully: the metric drop per missing feature stays under a budget, and the model never scores worse than the fallback baseline. Trade-off: imputation hides outages, so also alert on missing rates in monitoring. Measure: metric delta per feature and a handled-error test for unsupported patterns.

13.4 Behavioural Testing: Invariance, Directional and Metamorphic Tests

Conceptual Deep Dive

Aggregate metrics cannot tell you whether a model has learned the right *logic*. **Behavioural testing** (the CheckList approach of Ribeiro et al., 2020, built for NLP but applicable to tabular and other models) tests capabilities with three test types: **minimum functionality tests** (simple cases the model must get right), **invariance tests** (changing an irrelevant attribute must not change the output beyond a tolerance) and **directional expectation tests** (changing an input in a known direction must move the output in a known direction). Module 05 section 2 (legacy) shows the pytest form; this section adds structure and the reporting.

Metamorphic relations generalise this: a relation between outputs of related inputs that holds without needing the true label. Examples: scaling (doubling all quantities in a pricing feature scales the forecast), permutation (reordering an unordered list), monotonicity (higher income does not lower approval probability, all else equal), symmetry (swapping two interchangeable entities swaps their scores) and duplication (a duplicated row set gives the same aggregate prediction). Choose only relations a domain expert would agree with; a relation the business disputes is a debate, not a test.

Monotonic constraints can be both a *modelling* constraint (GBDT libraries such as XGBoost and LightGBM support per-feature monotonic constraints; check current docs for parameter names) and a *tested contract*: constrain in training, then verify with a directional test on the serialized artifact. **Counterfactual invariance** for fairness swaps protected-correlated tokens or attributes and asserts stable output, with caveats: proxies remain, and some attributes are legitimately informative, so this is a screening test, not proof of fairness (see chapter 17).

Test generation uses templates (fill slots from lexicons), perturbation of real rows, and **property-based testing** (the Hypothesis library) that generates inputs from a declared valid space. Report the **failure rate per behaviour** and track it across versions like a regression metric: "monotonicity in income: 0.4% violations", alerting on increases.

Edge cases: an invariance test fails legitimately because the feature really is informative (you declared the wrong invariance); a directional test fails in sparse regions of input space where the model extrapolates (restrict to supported ranges); and property tests generate invalid inputs the schema forbids (constrain the generator, otherwise failures mean nothing).

Cross-Connection Scenarios

- Slice tests use the same harness with cohorts instead of perturbations, see 13.5.
- Fairness-style invariance is a bridge to chapter 17.
- Monotonic constraints are chosen at training time, see 15.4 for tuning interactions.
- Violation rates feed the promotion checklist in 13.1.

Production Anti-Patterns

- Writing invariance tests for features the business actually wants to use: the suite blocks legitimate improvements and gets disabled.
- Asserting exact equality on floats: tests fail on harmless numeric differences and are ignored.
- Zero tolerance for violations: a 0.1% sparse-region violation blocks every release, so teams stop trusting the suite.

Minimal Working Code

A directional-expectation harness perturbs one feature on a batch and reports the violation fraction against a tolerance, on a hand-written scoring function with a deliberate bug at very high income.

```
import numpy as np

def score(X):                                # columns: income (k), debt (k), age
    income, debt, age = X[:, 0], X[:, 1], X[:, 2]
    z = 0.03 * income - 0.05 * debt + 0.01 * age - 1.0
    z = z - 0.08 * np.maximum(income - 150, 0)  # bug: penalises very high incomes
    return 1 / (1 + np.exp(-z))

def directional_violations(fn, X, col, factor, direction=+1, tol=1e-9):
    Xp = X.copy(); Xp[:, col] = Xp[:, col] * factor
    delta = fn(Xp) - fn(X)
```

```
    return float((direction * delta < -tol).mean())

rng = np.random.default_rng(0)
X = np.c_[rng.gamma(4, 15, 3000), rng.gamma(2, 10, 3000), rng.uniform(20, 70, 3000)]
rate = directional_violations(score, X, col=0, factor=1.10, direction=+1)
print(f"income +10% lowers the score for {rate:.2%} of rows")
assert rate <= 0.05, "directional test failed"              # tolerance agreed with the d
omain owner
strict = directional_violations(score, X, col=0, factor=1.10) == 0.0
print("zero-tolerance version passes:", strict)
```

A property-based style invariance test uses stdlib random generators to check that an order-insensitive model is permutation invariant (the Hypothesis equivalent is a comment) and that an order-sensitive one fails.

```
import math, random

def bag_model(items):                       # order-insensitive by construction
    return 1 / (1 + math.exp(-sum(items) / len(items)))

def seq_model(items):                       # accidentally order-sensitive: last item dominates
    return 1 / (1 + math.exp(-(items[-1] + 0.1 * sum(items[:-1]))))

def gen_case(rng):                          # Hypothesis: @given(lists(floats(-5, 5), min_size=2, max
_size=8))
    return [rng.uniform(-5, 5) for _ in range(rng.randint(2, 8))]

def check_permutation_invariance(model, n=300, seed=0, tol=1e-9):
    rng = random.Random(seed); bad = []
    for _ in range(n):
        items = gen_case(rng)
        shuffled = items[:]; rng.shuffle(shuffled)
        if abs(model(items) - model(shuffled)) > tol:
            bad.append(items)
    return bad

assert check_permutation_invariance(bag_model) == []
bad = check_permutation_invariance(seq_model)
print(f"seq_model violated invariance on {len(bad)} of 300 generated cases")
assert len(bad) > 100
```

Interview Angle

Q: Name three behavioural tests for a credit-scoring model and explain what failure each reveals. **A:** Directional: raising income (all else equal) must not lower approval probability; a failure reveals a spurious interaction or data artefact. Invariance: changing the applicant's name or an ID must not change the score; a failure reveals leakage or a memorised identifier. Minimum functionality: an applicant with zero debt and stable income must be approved above a probability; a failure reveals a floor problem. Trade-off: tolerance too tight gives noise. Measure: violation rate per behaviour, tracked per version.

Q: A directional test fails for 1.8% of rows. Bug or acceptable? **A:** Investigate where, not how many. Cluster the violating rows: if they sit in sparse regions or near feature extremes, it may be extrapolation noise; if they cluster in a segment, it may be a real bug or data artefact. Compare with the champion's rate and with the tolerance agreed with the domain owner. Trade-off: constraining the model (monotonic constraints) can cost a little accuracy. Measure: rate per segment, sample size per segment, and whether the new model's rate increased over the champion.

13.5 Slice-Based Testing, Error Analysis and Fairness-Aware Tests

Conceptual Deep Dive

An aggregate metric averages away the failures that hurt most. **Slices** are subsets defined by business segments (country, plan tier), device or region, data-quality flags (missing key feature), rare categories and intersections (new users on mobile). Discover them from error analysis rather than guessing: cluster misclassified rows by categorical feature combinations and look at the worst clusters (code below), or use embedding-based slice discovery for unstructured data.

Slice tests come in two forms: an absolute **slice floor** (the metric must not be confidently below a threshold) and a relative test against the champion (no slice may regress by more than a margin). Both need a minimum slice size and interval-aware comparison: with 30 rows a slice metric is mostly noise, so report "insufficient data" instead of pass or fail, and widen the slice or collect more data.

The **error analysis workflow** is a loop: confusion matrices by slice; top-loss examples inspected by a human; feature contribution of errors (which features differ between errors and correct cases); and a label audit, since high-loss examples are often mislabelled (see 2.16). The outcome is a targeted fix: data collection, relabelling, a feature, or an explicit fallback for the slice. Add each finding as a permanent slice and as regression cases (see 13.3).

Fairness-aware tests are a subset of slice tests where the slices are protected or sensitive groups: rate differences (selection rate, false positive rate), the equalized-odds gap, and calibration within group. They inherit the same statistical caveats (small groups, wide intervals) and the policy question of which metric is appropriate, which belongs to chapter 17.

The multiple-slice problem: with hundreds of slices some will fail by chance. Control it with false discovery rate procedures (Benjamini-Hochberg), an effect-size threshold in addition to a p-value, and a short list of gated slices (business-critical and protected) separate from an exploratory list that only warns. Automate slice reports in CI and publish them in the model card (see 4.16).

Edge cases: a slice too small for a stable estimate; a slice defined by a feature the model sees (circular, since the model can trivially overfit it); and a new category that appears only after launch (monitor for unseen values and route them to a catch-all slice).

Cross-Connection Scenarios

- Metric hierarchies and per-slice reporting begin in 1.16.
- Slice monitoring in production continues in 8.18.
- Fairness metrics and mitigation are in chapter 17.
- Error clusters feed targeted labelling in 2.16.

Production Anti-Patterns

- Reporting only the global metric: one cohort degrades for months because the average stays flat.
- Treating hundreds of slices at p<0.05 as findings: the team chases noise and loses trust in the report.
- A slice list never updated after incidents: the next incident happens in the same unlisted segment.

Minimal Working Code

A slice-metric table returns accuracy, n and a bootstrap interval per slice, and flags a slice only when it is below the floor with confidence.

```
import numpy as np

def slice_table(correct, slices, floor, n_boot=500, seed=0, min_n=30):
    rng = np.random.default_rng(seed); rows = []
    for s in sorted(set(slices)):
        c = correct[slices == s]; n = len(c)
        if n < min_n:
            rows.append((s, n, None, None, None, "insufficient data")); continue
        boots = rng.choice(c, size=(n_boot, n)).mean(axis=1)
        lo, hi = np.quantile(boots, [0.025, 0.975])
        rows.append((s, n, round(float(c.mean()), 3), round(float(lo), 3), round(float(hi), 3),
                     "BELOW FLOOR" if hi < floor else "ok"))
    return rows

rng = np.random.default_rng(1)
slices = np.array(["desktop"] * 1500 + ["mobile"] * 500 + ["tablet"] * 12)
p_correct = np.where(slices == "mobile", 0.72, 0.92)
correct = (rng.random(len(slices)) < p_correct).astype(float)
print("global accuracy", round(float(correct.mean()), 3))
rows = slice_table(correct, slices, floor=0.85)
for r in rows:
    print(r)
assert {r[0]: r[5] for r in rows} == {"desktop": "ok", "mobile": "BELO
W FLOOR", "tablet": "insufficient data"}
```

An error-cluster finder groups misclassified rows by feature combinations and ranks them by excess error rate with a minimum support.

```
from collections import Counter
from itertools import combinations
import random

def error_clusters(rows, features, global_err, min_support=40, max_order=2):
    total, errs = Counter(), Counter()
    for r in rows:
        for k in range(1, max_order + 1):
            for combo in combinations(features, k):
                key = tuple((f, r[f]) for f in combo)
                total[key] += 1; errs[key] += r["err"]
    out = [(key, total[key], errs[key] / total[key] - global_err) for key in total if total[key] >= mi
n_support]
    return sorted(out, key=lambda x: -x[2])

rng = random.Random(0)
rows = []
for _ in range(4000):
    device, region, plan = rng.choice(["ios", "android", "web"]), rng.choic
e(["us", "eu", "apac"]), rng.choice(["free", "pro"])
    p_err = 0.45 if (device == "android" and region == "eu") else 0.08          # pla
nted problem
    rows.append({"device": device, "region": region, "plan": plan, "er
r": int(rng.random() < p_err)})
g = sum(r["err"] for r in rows) / len(rows)
top = error_clusters(rows, ["device", "region", "plan"], g)[:3]
for key, n, excess in top:
    print(dict(key), n, f"excess error {excess:+.2f}")
assert dict(top[0][0]) == {"device": "android", "region": "eu"}
```

Interview Angle

Q: Aggregate AUC is flat but the mobile cohort collapsed. How should the test suite have caught it? **A:** A gated slice floor on mobile with an interval-aware comparison against the champion would fail the build, provided the slice was on the list and big enough. Also monitor per-slice metrics in production and use error-cluster analysis to discover slices not yet listed. Trade-off: more slices mean more false alarms, so gate only business-critical and protected slices and control the rest with FDR. Measure: coverage (share of traffic in gated slices) and time-to-detect for a seeded slice regression.

Q: How do you choose which slices to gate on? **A:** Start from business impact (revenue or risk share), protected groups required by policy, historical incident segments, and data-quality flags; then keep only slices large enough for a stable interval. Add slices after each incident and retire dead ones. Gate on a floor plus a relative-to-champion margin; leave exploratory slices as warnings. Trade-off: too many gated slices block releases on noise. Measure: share of traffic covered, gate false-positive rate and escaped slice incidents.

13.6 Pipeline, Serving, Load and Production Testing

Conceptual Deep Dive

Models fail through their plumbing at least as often as through their weights. **Pipeline integration tests** run the whole DAG end to end on a tiny dataset in CI: ingest, validate, featurize, train, evaluate, register. Add **idempotency** tests (run twice on the same input, expect identical output; see 5.16), backfill tests (re-run a past partition and compare) and **failure injection** (kill the k-th task, expect a safe retry or a clean abort, never a half-written output).

Serving contract tests pin the request and response schema, version header, error codes, and a set of golden requests with expected responses; they guarantee backward compatibility for consumers and fail when a model change alters the output type. **Load tests** (and soak tests) measure p95/p99 latency at target RPS, error rate, memory growth over hours and cold-start time. Use realistic payload distributions: uniform inputs make batching and caches look better than reality.

Replay testing sends logged production requests, with the features *as logged*, to a candidate and compares outputs and latency with the incumbent; it is the best predictor of behaviour on real traffic, and a good way to test serving code changes. Replaying

features recomputed now instead of as logged hides skew. Scrub or synthesise PII before replay data enters test environments.

Chaos and degradation tests simulate the feature store being down, a slow dependency, a malformed payload and a model-load failure, and verify that fallback tiers engage (cached features, default values, a simpler model, a rules path; see Module 06 and Module 11 (legacy)). Assert the *behaviour*: latency bounded, correct status code, a metric emitted.

Testing in production is a controlled, partial form of the same idea: shadow deployments (candidate sees traffic, answers are discarded), canary analysis with automatic abort (see Module 07 (legacy)), synthetic probes with known answers, and monitors treated as continuous tests. Keep the contract: anything a test environment cannot exercise, a probe or monitor must.

A pre-release checklist for a model version goes beyond offline metrics: data and model suites green, slice and behavioural tests unchanged, idempotent pipeline, contract and replay tests passing, load test within SLO, degradation test engaged, rollback verified (see chapter 7), and monitors and dashboards updated for the new version.

Edge cases: a load test with uniform inputs overstates batching gains; a replay using recomputed features hides skew; and idempotency broken by nondeterministic ordering (unsorted dict or set iteration, parallel writers).

Cross-Connection Scenarios

- Atomic publish and idempotent backfills are detailed in 5.16.
- Tail latency and load shedding are in 6.17.
- Shadow and canary mechanics are in Module 07 (legacy).
- Monitors as continuous tests continue in chapter 8.

Production Anti-Patterns

- Load testing only the happy path: the first slow dependency or malformed payload triggers a cascade that was never rehearsed.
- Testing the pipeline only in production: a bug in a backfill corrupts live tables before anyone sees it.
- Replaying production PII into test environments: a privacy incident created by the test process itself.

Minimal Working Code

A tiny asyncio load generator sends concurrent requests to a stub handler and reports p50/p95/p99 and error rate.

```
import asyncio, random, time

def make_handler(n, seed=0):
    rng = random.Random(seed)
    delays = [0.002 + rng.expovariate(1 / 0.004) for _ in range(n)]       # skewed, not uniform
    async def handler(i):
        await asyncio.sleep(delays[i])
        if i % 50 == 49:
            raise RuntimeError("upstream error")
    return handler

async def run_load(n=200, concurrency=20):
    handler, sem, results = make_handler(n), asyncio.Semaphore(concurrency), []
    async def one(i):
        async with sem:
            t = time.perf_counter(); ok = True
            try:
                await handler(i)
            except RuntimeError:
                ok = False
            results.append(((time.perf_counter() - t) * 1000, ok))
    await asyncio.gather(*(one(i) for i in range(n)))
    lat = sorted(r[0] for r in results)
    q = lambda p: lat[int(p * (len(lat) - 1))]
    return {"p50": q(0.5), "p95": q(0.95), "p99": q(0.99), "error_rate
": sum(not r[1] for r in results) / n}

report = asyncio.run(run_load())
print({k: round(v, 3) for k, v in report.items()})
```

```
assert report["error_rate"] == 0.02 and report["p50"] <= report["p99"]
```

An idempotency helper runs a pipeline twice in separate temp dirs and compares bytes; a failure-injection wrapper raises on the k-th call and the atomic publish leaves no partial output.

```
import json, os, tempfile

def pipeline(records, out_dir, transform=lambda r: {**r, "v": r["v"] * 2}):
    rows = sorted((transform(r) for r in records), key=lambda r: r["id"])      # sorting remove
s order dependence
    tmp, final = os.path.join(out_dir, "out.json.tmp"), os.path.join(out_dir, "out.json&qu
ot;)
    with open(tmp, "w") as f:
        json.dump(rows, f, sort_keys=True)
    os.replace(tmp, final)                                          # atomic publish

def read_bytes(d):
    with open(os.path.join(d, "out.json"), "rb") as f:
        return f.read()

def check_idempotent(records):
    with tempfile.TemporaryDirectory() as a, tempfile.TemporaryDirectory() as b:
        pipeline(records, a); pipeline(list(reversed(records)), b)              # same data, different ar
rival order
        assert read_bytes(a) == read_bytes(b), "pipeline is not idempotent"

def fail_on_kth(fn, k):
    calls = {"n": 0}
    def wrapped(*args):
        calls["n"] += 1
        if calls["n"] == k:
            raise RuntimeError(f"injected failure on call {k}")
        return fn(*args)
    return wrapped

records = [{"id": i, "v": i} for i in range(5)]
check_idempotent(records)
with tempfile.TemporaryDirectory() as d:
    try:
        pipeline(records, d, transform=fail_on_kth(lambda r: {**r, "v": r["v"] * 2},
3))
    except RuntimeError as e:
        print("failed as injected:", e)
    assert not os.path.exists(os.path.join(d, "out.json")), "partial output was published&
quot;
    pipeline(records, d)                                            # clean rerun succeeds
    print("rerun ok, bytes:", len(read_bytes(d)))
```

Interview Angle

Q: How do you test a pipeline for idempotency and safe backfill? **A:** Run the pipeline twice on identical input (and once with shuffled arrival order) and assert byte-identical outputs; publish atomically so partial runs are invisible; write backfills to a new partition or table and compare with existing values before swapping; inject a failure at each task and assert retries converge to the same result. Trade-off: determinism needs sorted keys and fixed seeds, which can cost performance. Measure: byte-diff count across reruns (target zero) and the fraction of tasks covered by failure injection.

Q: What would your pre-release checklist for a new model version contain beyond offline metrics? **A:** Contract tests with golden requests, replay of logged traffic comparing outputs and latency, a load and soak test at target RPS with realistic payloads, a degradation test showing fallbacks engage, a verified rollback path, a shadow period with divergence under a threshold, and updated monitors and runbooks. Trade-off: each step lengthens release time, so tier checks by risk. Measure: p99 within SLO, replay divergence below an agreed bound, and rollback time in a drill.